

APPLIED MATH 3

WEEK 6

ROTATION “FUN”-DAMENTALS I*

* Not guaranteed to be fun

ADD A COUPLE MORE EXAMPLES HERE.

- Dealing with rotations in Unreal can be tricky.
- You will encounter rotation problems constantly however so it is important to understand the tools you have to work with them.
- Just like vector math is critical to moving objects, understanding trigonometry is important for rotation of objects.

UNREAL ROTATORS DEFINITION

- The Rotator object defines a single rotation in 3D space.
- The rotation data in a Transform is a Rotator for example.
- You can also create Rotator objects by defining their 3 axes of rotation individually.

UNREAL ROTATORS DEFINITION

- Dealing with rotations in Unreal can be tricky, particularly because the documentation is terrible.
- There is an extensive description of a Rotator in the Unreal documentation. It reads:
 - “Rotator”
 - Yes, that’s it. Document your code kids. (Does include methods though)
 - <https://docs.unrealengine.com/4.27/en-US/BlueprintAPI/Math/Rotator/>
 - Some actual docs here:
 - <https://docs.unrealengine.com/4.26/en-US/BuildingWorlds/LevelEditor/Details/Properties/VectorAndRotator/>

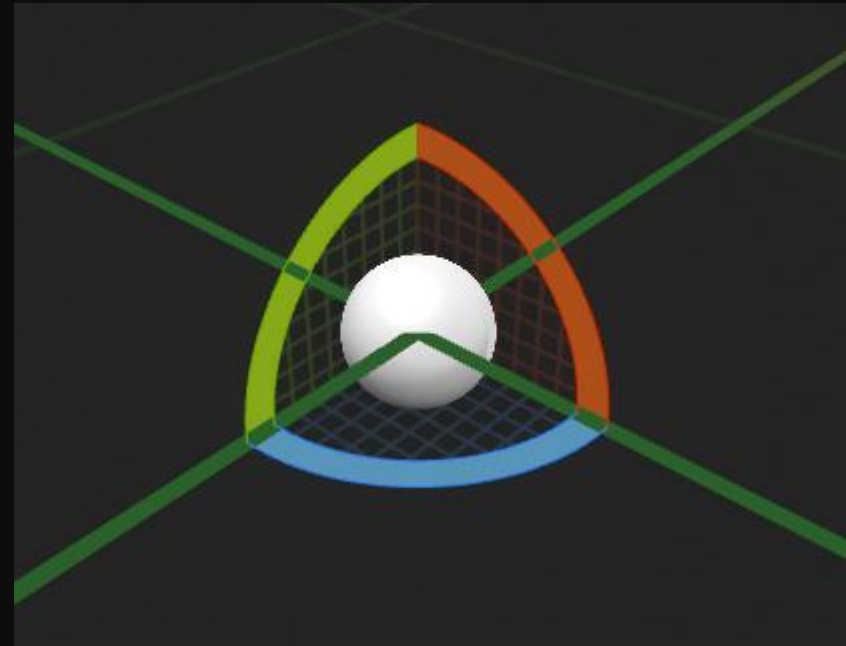
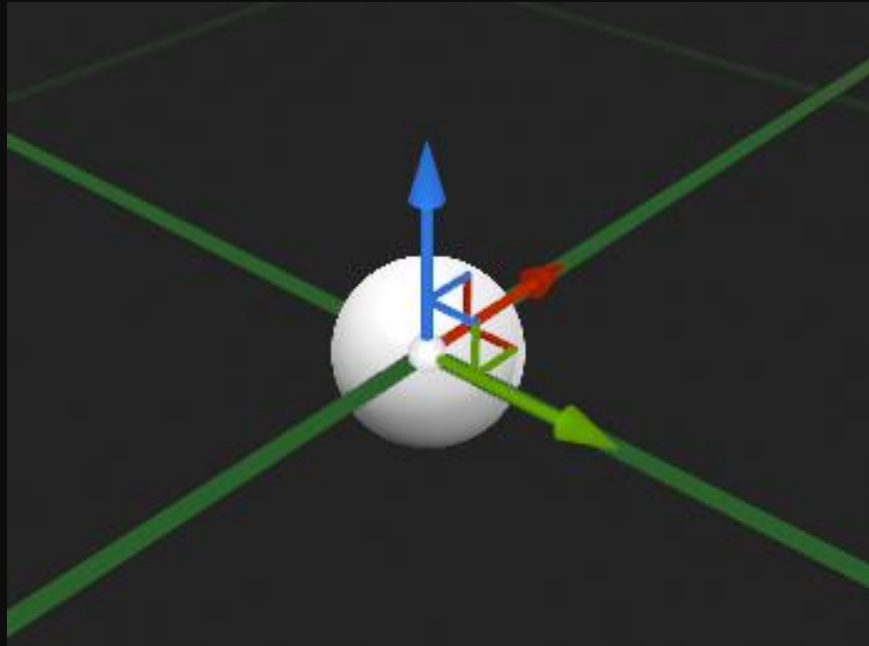
ROTATION NOTATION: EULER ANGLES

- A Rotation object in Unreal is a struct (data structure) that stores rotation data.
- On the surface (ie in blueprint, in transforms) a Rotation is stored using a 3D Vector of Euler angles. These are the typical angle measurements you have seen in trigonometry problems before.
- Eulers are the familiar X, Y, Z angles stored in degrees and have a range from $(0^\circ, 360^\circ)$. If you enter an angle greater or less than this range, it will get mapped to the angle within the range (it wraps around basically).
- For example 720° is the same value as 360° , or 540° is the same as 180° .
 - *This can cause issues when you are rotating from a Rotation to another, as it may “wrap” the wrong way.

ROTATION NOTATION: EULER

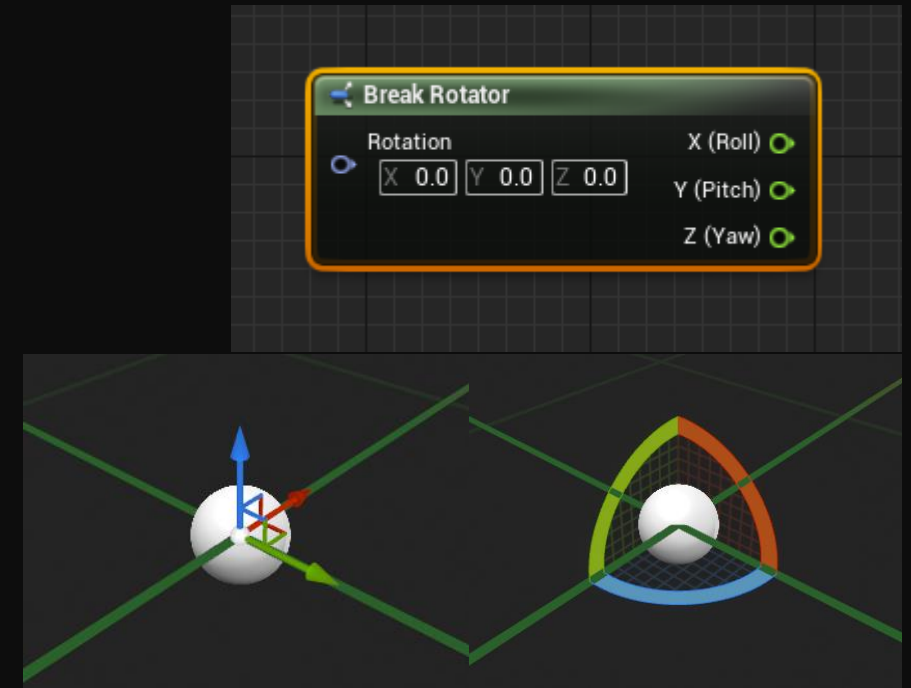
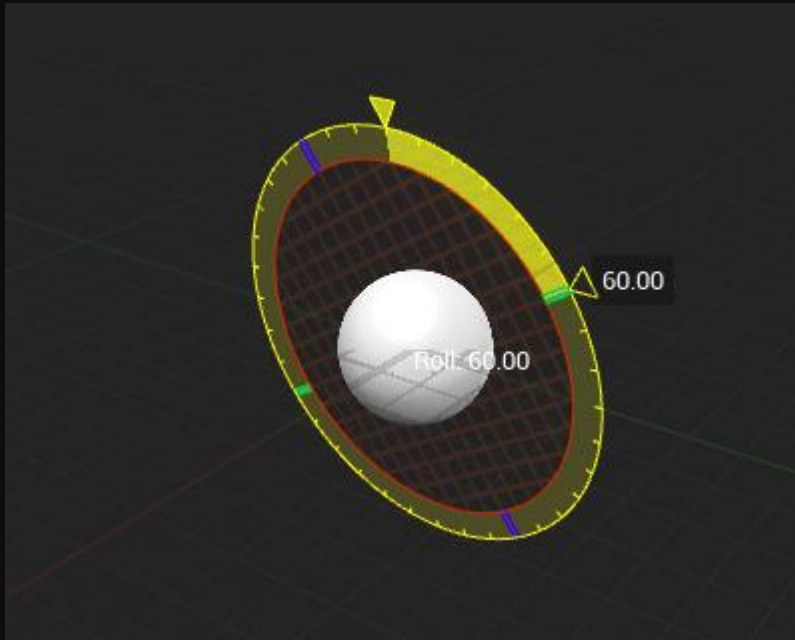
- Euler Rotation Format stores the rotation as 3 Euler angles.
- Stored as three separate rotation values, one for each axis (X, Y, Z).
- Recall: When moving (in Unreal, in 3D):
 - X Axis represents Forward
 - Y Axis represents Right
 - Z Axis represents Up
- The rotation axes rotate around the same axis the movement occurs in.
 - X Axis Rotation represents Roll (Tilt your head from side to side)
 - Y Axis rotation represents Pitch (Nod your head)
 - Z Axis rotation represents Yaw (Turn your head to look left and right)

ROTATION NOTATION



ROTATION NOTATION

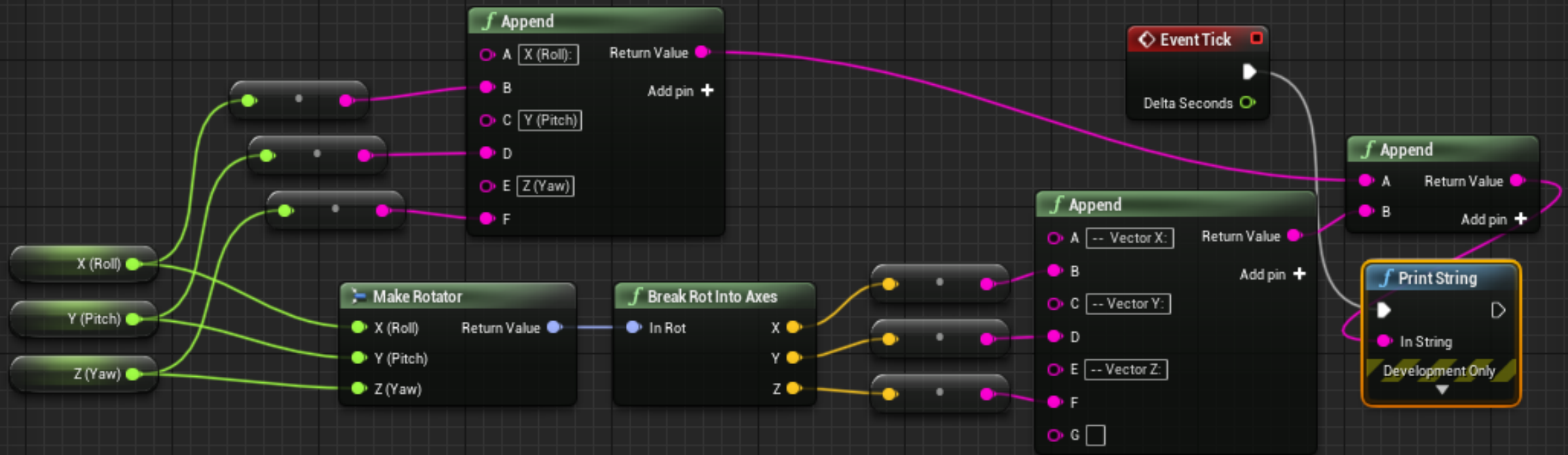
- If you imagine facing in the direction of a given movement axis, a positive rotation in that axis will rotate it clockwise by degrees. A negative rotation in that axis will rotate counter-clockwise.



ROTATION NOTATION: VECTOR AXIS OF ROTATION

- We can also convert the Rotation object to a series of Vectors that represent the amount of each direction that is used.

ROTATION NOTATION: VECTOR AXIS OF ROTATION



ROTATION NOTATION: VECTOR AXIS OF ROTATION

- The output we get from the previous slide is interesting. Here are some examples of different input values.
- The X,Y,Z input is expressed in Degrees, as an angle.
- In this first example, the Vectors produced are unit vectors that point in the X, Y and Z direction.

X (Roll): 0.0Y (Pitch)0.0Z (Yaw)0.0-- Vector X: X=1.000 Y=0.000 Z=0.000-- Vector Y: X=0.000 Y=1.000 Z=-0.000-- Vector Z: X=-0.000 Y=0.000 Z=1.000

ROTATION NOTATION: VECTOR AXIS OF ROTATION

- The second example is similar, except the Y vector now points in the $-Z$ direction, and the Z points in the Y direction. This is due to it being rotated by 90° . The X Vector is unchanged because we are rotating our rotator AROUND the X axis.

X (Roll): 0.0Y (Pitch)0.0Z (Yaw)0.0-- Vector X: X=1.000 Y=0.000 Z=0.000-- Vector Y: X=0.000 Y=1.000 Z=-0.000-- Vector Z: X=-0.000 Y=0.000 Z=1.000

X (Roll): 90.0Y (Pitch)0.0Z (Yaw)0.0-- Vector X: X=1.000 Y=0.000 Z=0.000-- Vector Y: X=0.000 Y=-0.000 Z=-1.000-- Vector Z: X=-0.000 Y=1.000 Z=-0.000

ROTATION NOTATION: VECTOR AXIS OF ROTATION

- The third example is the most interesting. Where have you seen the number 0.707 expressed before?

X (Roll): 0.0Y (Pitch)0.0Z (Yaw)0.0-- Vector X: X=1.000 Y=0.000 Z=0.000-- Vector Y: X=0.000 Y=1.000 Z=-0.000-- Vector Z: X=-0.000 Y=0.000 Z=1.000

X (Roll): 90.0Y (Pitch)0.0Z (Yaw)0.0-- Vector X: X=1.000 Y=0.000 Z=0.000-- Vector Y: X=0.000 Y=-0.000 Z=-1.000-- Vector Z: X=-0.000 Y=1.000 Z=-0.000

X (Roll): 45.0Y (Pitch)0.0Z (Yaw)0.0-- Vector X: X=1.000 Y=0.000 Z=0.000-- Vector Y: X=0.000 Y=0.707 Z=-0.707-- Vector Z: X=-0.000 Y=0.707 Z=0.707

ROTATION NOTATION: VECTOR AXIS OF ROTATION

- The third example is the most interesting. Where have you seen the number 0.707 expressed before?
 - (Hint: In Trigonometry)

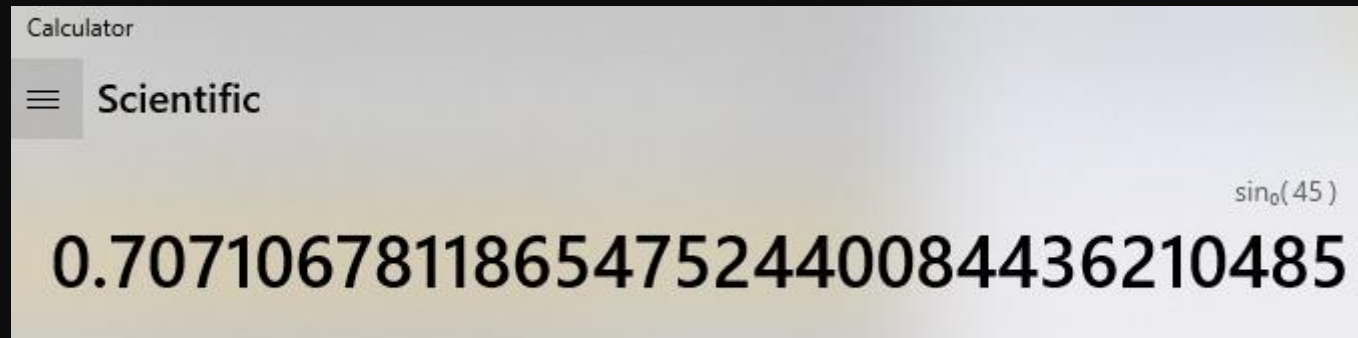
X (Roll): 0.0Y (Pitch)0.0Z (Yaw)0.0-- Vector X: X=1.000 Y=0.000 Z=0.000-- Vector Y: X=0.000 Y=1.000 Z=-0.000-- Vector Z: X=-0.000 Y=0.000 Z=1.000

X (Roll): 90.0Y (Pitch)0.0Z (Yaw)0.0-- Vector X: X=1.000 Y=0.000 Z=0.000-- Vector Y: X=0.000 Y=-0.000 Z=-1.000-- Vector Z: X=-0.000 Y=1.000 Z=-0.000

X (Roll): 45.0Y (Pitch)0.0Z (Yaw)0.0-- Vector X: X=1.000 Y=0.000 Z=0.000-- Vector Y: X=0.000 Y=0.707 Z=-0.707-- Vector Z: X=-0.000 Y=0.707 Z=0.707

ROTATION NOTATION: VECTOR AXIS OF ROTATION

- The third example is the most interesting. Where have you seen the number 0.707 expressed before?

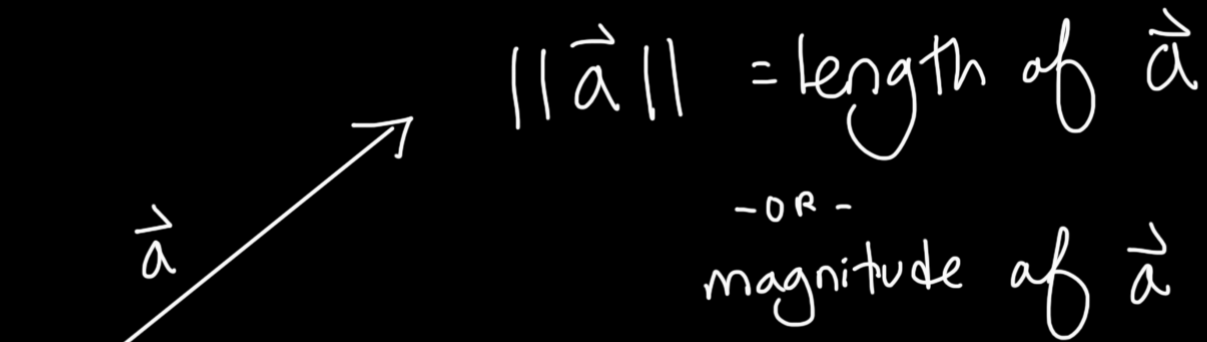


X (Roll): 0.0Y (Pitch)0.0Z (Yaw)0.0-- Vector X: X=1.000 Y=0.000 Z=0.000-- Vector Y: X=0.000 Y=1.000 Z=-0.000-- Vector Z: X=-0.000 Y=0.000 Z=1.000

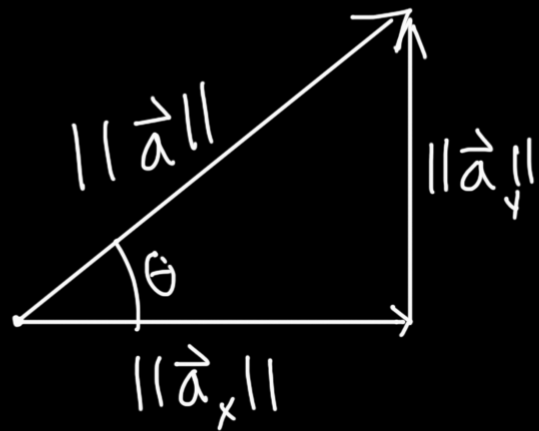
X (Roll): 90.0Y (Pitch)0.0Z (Yaw)0.0-- Vector X: X=1.000 Y=0.000 Z=0.000-- Vector Y: X=0.000 Y=-0.000 Z=-1.000-- Vector Z: X=-0.000 Y=1.000 Z=-0.000

X (Roll): 45.0Y (Pitch)0.0Z (Yaw)0.0-- Vector X: X=1.000 Y=0.000 Z=0.000-- Vector Y: X=0.000 Y=0.707 Z=-0.707-- Vector Z: X=-0.000 Y=0.707 Z=0.707

VECTORS AND TRIG: WHO CARES ANYWAY?



VECTORS AND TRIG: THEY ARE THE SAME!



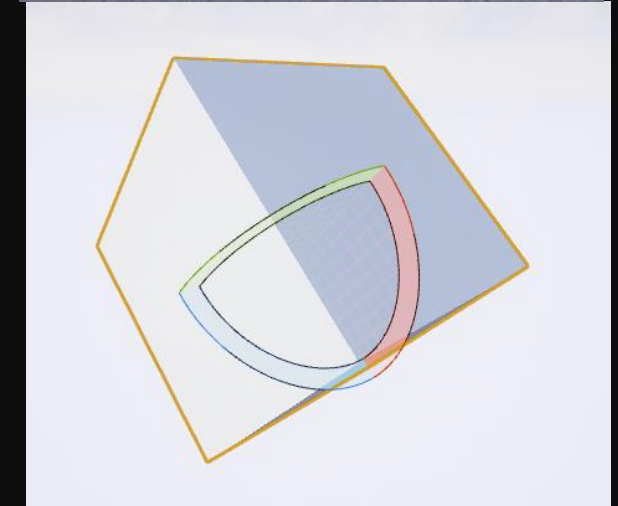
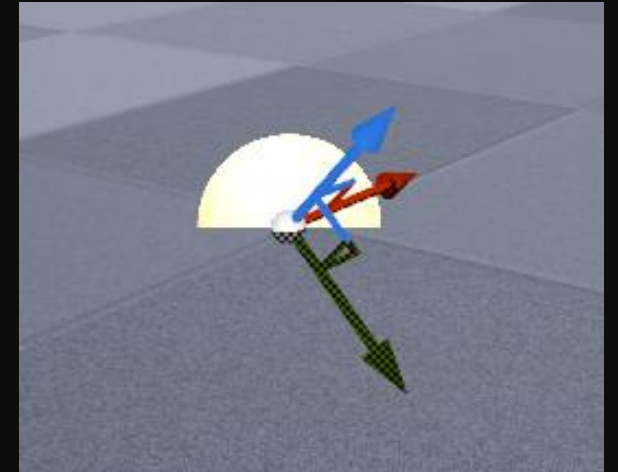
we have created
a right angle triangle!

$$\sin \theta = \frac{\|\vec{a}_y\|}{\|\vec{a}\|}$$

$$\cos \theta = \frac{\|\vec{a}_x\|}{\|\vec{a}\|}$$

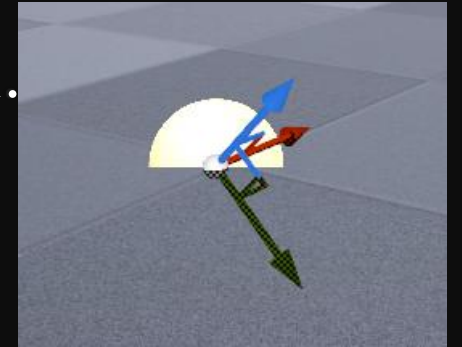
2D TO 3D TRIG: ROTATED 45° ON X AXIS

- As you can see in the diagram, we are seeing 0.707 as the Y and Z axis are basically at a 45° angle to the world axis.
- This is to say, everything we learned about Sine and Cosine holds up in 3 dimensions!
- We just have to be careful about which axis we are referencing as the base.



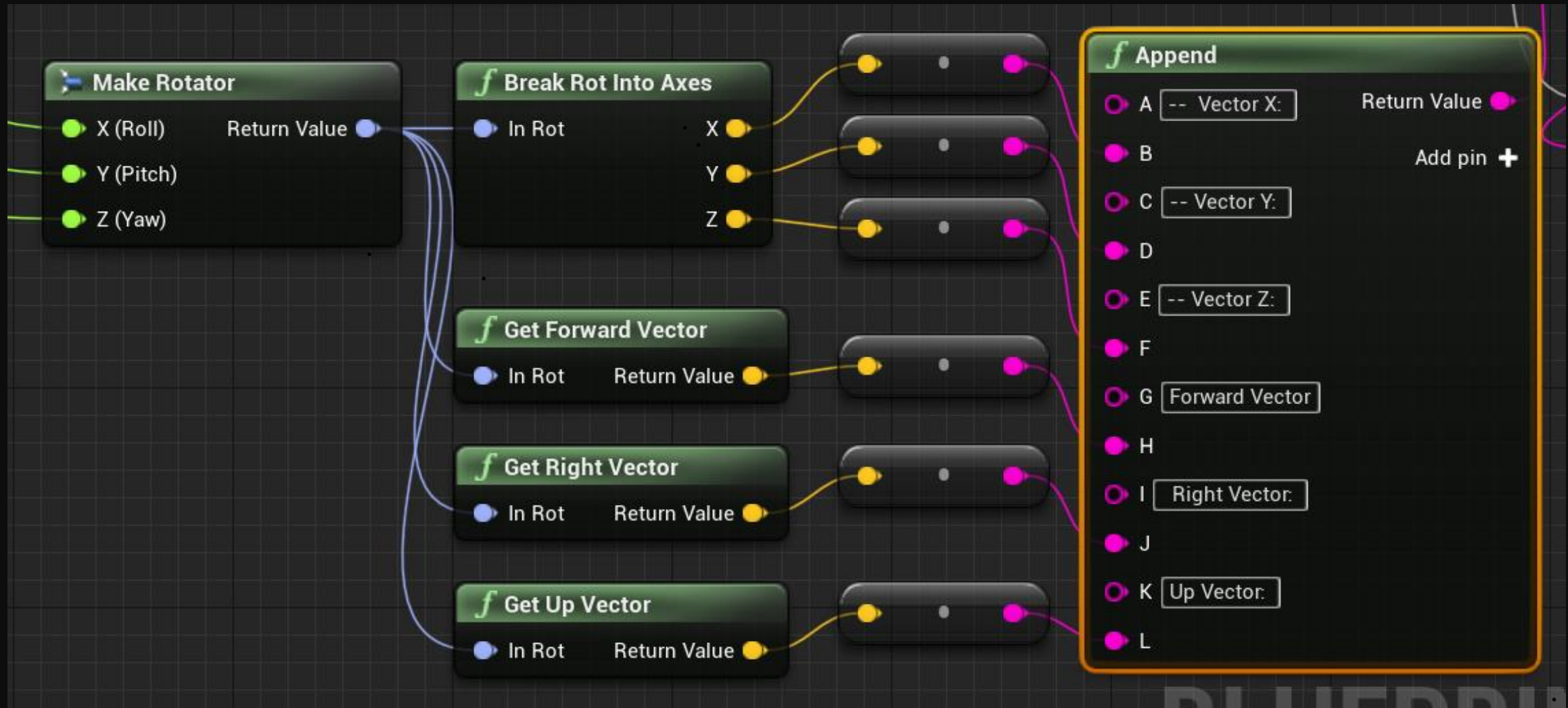
ROTATION NOTATION: VECTOR AXIS OF ROTATION

- Look at our data again for our object rotated 45° on X.
- The X Vector red ($X = 1.0, y = 0, z = 0$)
- The Y Vector is green ($X = 0, Y = 0.707, Z = -0.707$)
- The Z Vector is blue ($X = 0, Y = 0.707, Z = 0.707$)
- So what we have done is convert our Rotation orientation into its three component unit vectors.



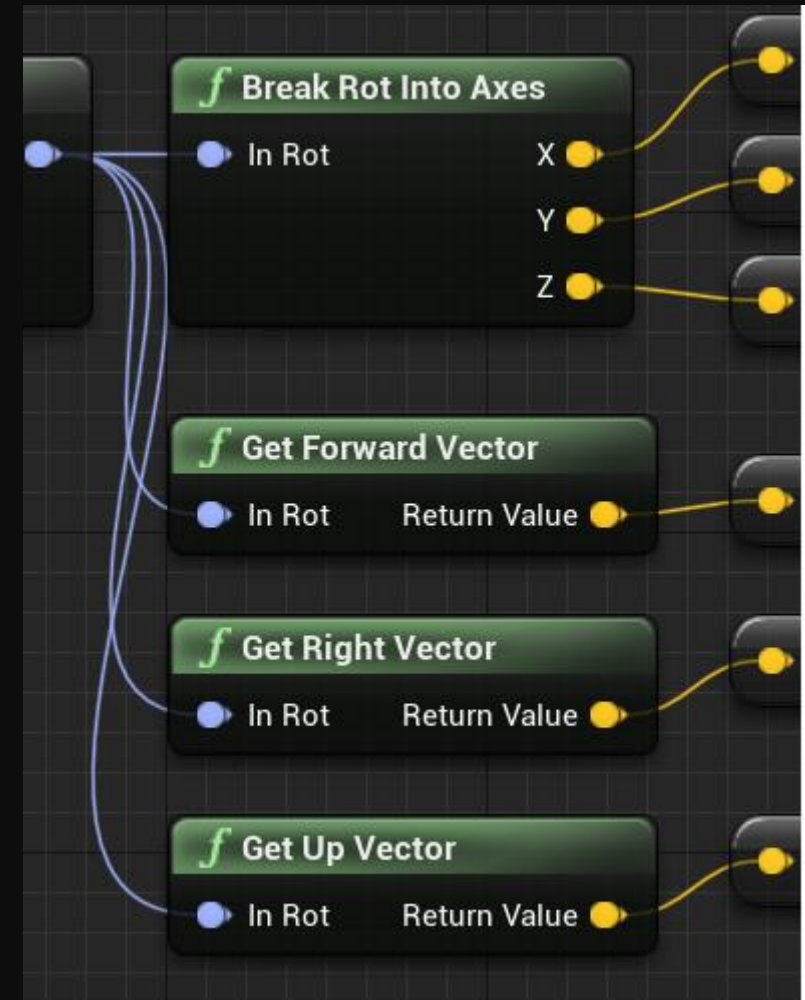
X (Roll): 45.0Y (Pitch)0.0Z (Yaw)0.0-- Vector X: X=1.000 Y=0.000 Z=0.000-- Vector Y: X=0.000 Y=0.707 Z=-0.707-- Vector Z: X=-0.000 Y=0.707 Z=0.707

NOTE: THESE DO EXACTLY THE SAME THING!



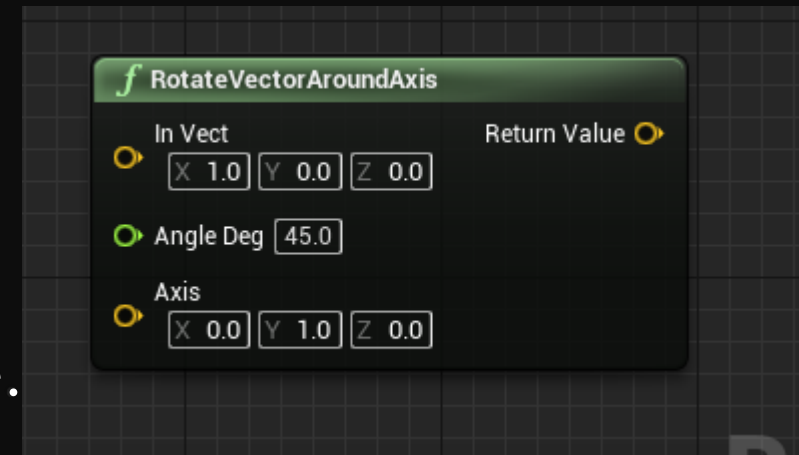
ROTATION NOTATION: VECTOR AXIS OF ROTATION

- The X Axis from the Break is the same as the Forward Vector
- The Y is the same as the Right Vector
- The Z Axis is the same as the Up Vector



ROTATION NOTATION: VECTOR AXIS OF ROTATION

- It is important to understand the different ways a rotation can be stored as you will sometimes need to use each.
- For example, when you want to rotate a Vector around a specified axis, you can use the **RotateVectorAroundAxis** node, which specifies both the vector to rotate, as well as the axis to rotate it around.
- HOWEVER, it requires the axis in the form of a vector, so you will need to be able to understand what Unreal is asking for to be able to use this node.



ROTATION NOTATION: QUATERNIONS

- While we have talked a lot about the Rotation object type, you should be aware that internally the data is stored as a Quaternion.
- Quaternions are 4 dimensional data types that you don't normally work with directly or worry about. They are used because they avoid several problems that occur when trying to rotate 3 dimensional Euler angles.
- For now, I am just going to hand wave it away, but we may touch on it later. (If you are interested in researching it yourself, we are trying to avoid "Gimbal Lock")

TODO:

- In-Class challenges:
- Rotate your Pawn around a given axis (such as the Z axis) when input is applied.
- Rotate your pawn art component to face a given target (will require the vector math from last week as well).
 - Hint: if you don't want to spin wildly you will have to control the axis.
- Rotate your pawn in 45° increments when you add input.
 - Hard Mode: Rotate towards a target at a fixed speed.
 - Hard Mode 2: Rotate towards a target, easing in and out of the movement.